

UNIVERSIDAD
NACIONAL
DE COLOMBIA

Departamento de Ingeniería Mecánica y Mecatrónica

IA y Minirobots

Taller 6. Machine Learning

Autores:

Yeison Esteban Ortega Chacua

Daniel Alberto Rodriguez Porras

Daniel Santiago Navarro Gil

Docente:

José Martínez

Bogotá D.C, Colombia

Desarrollo

1 Aprendizaje de la multiplicación de matrices

Planteamiento y procedimiento

Se generaron 50.000 pares de matrices de 2×2 , con entradas enteras aleatorias entre -20 y 20 . Cada registro contiene ocho entradas y cuatro salidas calculadas analíticamente. Se destinaron 40.000 registros al entrenamiento y 10.000 a prueba, sin fijar una semilla aleatoria.

Para $A = [[a, b], [c, d]]$ y $B = [[e, f], [g, h]]$, el producto es $AB = [[ae + bg, af + bh], [ce + dg, cf + dh]]$. Se empleó regresión lineal sobre características polinómicas de grado 2, porque una regresión aplicada únicamente a las ocho entradas no puede representar sus productos. La expansión produce 45 características; sus coeficientes se aprendieron con los datos, sin asignar manualmente la fórmula.

Resultados y análisis

Indicador	Resultado en prueba
MAE	$2,785 \times 10^{-13}$
RMSE	$3,643 \times 10^{-13}$
R^2	1,000000000000
Error absoluto máximo	$2,018 \times 10^{-12}$
Tiempo de entrenamiento	0,4412 s

El modelo recuperó las cuatro expresiones del producto, con coeficientes relevantes iguales a 1. En diez pares nuevos, las predicciones coincidieron con el resultado analítico con tolerancia absoluta de 10^{-8} . Por ejemplo, $A = [[11, -16], [-17, 9]]$ y $B = [[4, -11], [18, -7]]$ produjeron $[[-244, -9], [94, 124]]$, con error máximo de $8,527 \times 10^{-13}$. Las métricas se calcularon sin redondear las predicciones.

Método	Un par ms por llamada	10.000 pares ms por llamada
Fórmula analítica	0,018456	0,283091
NumPy A @ B	0,003649	3,524056
Modelo ML completo	0,454989	16,449850

El modelo tardó 24,65 veces más que la fórmula para un par y 58,11 veces más para el lote, además del entrenamiento. El cálculo directo necesita 8 multiplicaciones y 4 sumas; la evaluación densa del modelo requiere aproximadamente 216 multiplicaciones y 176 sumas, incluida la expansión. Los tiempos dependen del entorno y del tamaño del lote. Para esta operación conocida conviene el cálculo analítico; el ejercicio demuestra que un modelo apropiado puede aprender la relación a partir de ejemplos.

2 Clasificación de registros sísmicos mediante SVM

Planteamiento

Se adaptó el notebook de SVM suministrado para clasificar registros del catálogo del U.S. Geological Survey (USGS), entidad gubernamental estadounidense. Se definieron dos clases: profundidad menor de 70 km y profundidad de 70 km o más. Se trata de una clasificación retrospectiva; el modelo no predice la ocurrencia de futuros sismos ni su peligrosidad.

La consulta incluyó los primeros 3.000 eventos, ordenados cronológicamente, de una búsqueda mundial con magnitud mínima 4 entre enero de 2025 y enero de 2026. El límite se alcanzó el 5 de marzo de 2025, por lo que no se analizó el año completo. No se excluyeron registros durante la limpieza. La clase menor de 70 km contiene 2.206 casos y la clase de 70 km o más, 794.

Fundamento y adaptación del algoritmo

SVM aprende una frontera entre clases procurando un margen amplio. Los vectores de soporte son ejemplos que participan en su definición. El parámetro C controla la penalización de las violaciones del margen; con RBF, gamma controla el alcance de la influencia de cada ejemplo. Una frontera lineal puede ser insuficiente cuando las clases presentan relaciones curvas, como ilustra la demostración sintética de dos lunas.

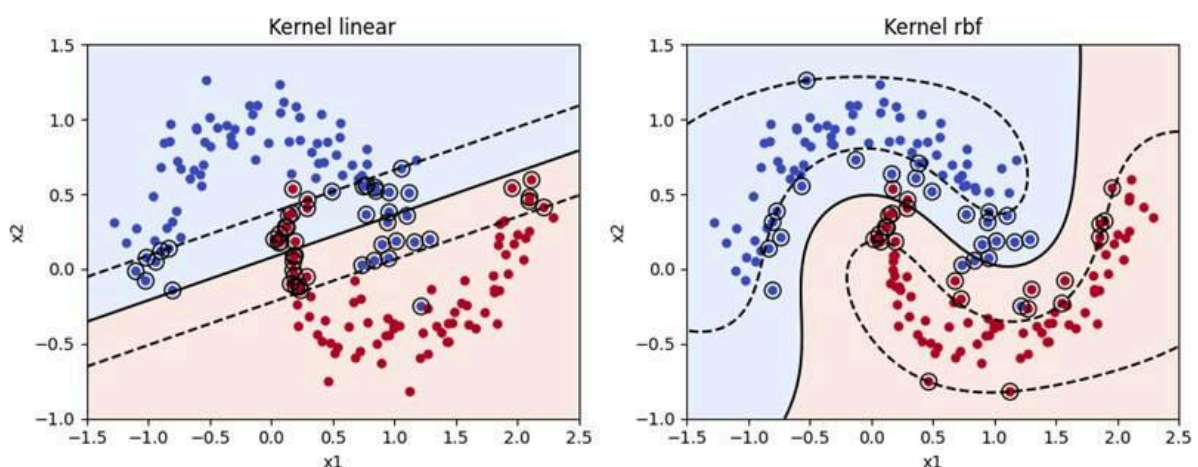


Figura 1. Demostración sintética de kernels lineal y RBF. Los círculos identifican vectores de soporte.

Las entradas de la aplicación fueron la magnitud y tres coordenadas derivadas de latitud y longitud sobre una esfera unitaria, evitando la discontinuidad entre -180° y $+180^\circ$. La profundidad se utilizó exclusivamente para crear la etiqueta y comprobar los resultados; incluirla como entrada habría revelado directamente la respuesta.

Se reservaron los últimos 600 registros para prueba y se usaron los primeros 2.400 para entrenamiento. Dentro de estos se aplicó validación temporal con tres particiones. Una Pipeline integró StandardScaler y SVC, de modo que el escalamiento se ajustó solo con el entrenamiento de cada partición. Se utilizó ponderación balanceada para compensar las frecuencias diferentes de las clases.

Se evaluaron siete configuraciones: kernel lineal con $C = 0,1; 1; 10$, y RBF con $C = 1; 10$ y $\gamma = \text{scale}; 1$. La elección se basó en F1 macro de validación, sin consultar el conjunto final de prueba. Se seleccionó RBF con $C = 1$ y $\gamma = 1$; su F1 macro de validación fue 0,713. Tras el ajuste final hubo 862 y 287 vectores de soporte de las clases 0 y 1, respectivamente.

Resultados de la clasificación y discusión

En los 600 registros de prueba se obtuvieron 379 aciertos y 221 errores. Los primeros diez ejemplos mostrados tuvieron nueve aciertos, pero la evaluación válida corresponde a todo el conjunto de prueba. Las filas de la matriz representan la clase real y las columnas, la predicción.



Figura 2. Matriz de confusión obtenida en los registros reservados para prueba.

Clase	Precisión	Recall	F1	Casos
Menor de 70 km	0,854	0,523	0,649	390
70 km o más	0,485	0,833	0,613	210

La exactitud global fue 63,17 %, la exactitud balanceada 67,82 % y el F1 macro 0,631. Se identificaron 175 de los 210 casos de 70 km o más, pero 186 de los 390 casos de menor profundidad se clasificaron erróneamente en ese grupo. Por tanto, la alta sensibilidad de la clase 1 se acompañó de baja precisión: solo el 48,5 % de sus predicciones fue correcto.

La referencia que siempre predice “menor de 70 km” obtiene 65,0 % de exactitud, 50,0 % de exactitud balanceada y F1 macro de 0,394. Aunque el SVM tiene menor exactitud global, mejora las medidas que consideran ambas clases. El F1 macro de entrenamiento fue 0,759 frente a 0,631 en prueba; la caída indica menor desempeño en los eventos posteriores y puede relacionarse con sobreajuste o cambios en la distribución.

3 . K-NEAREST NEIGHBORS (KNN)

Planteamiento

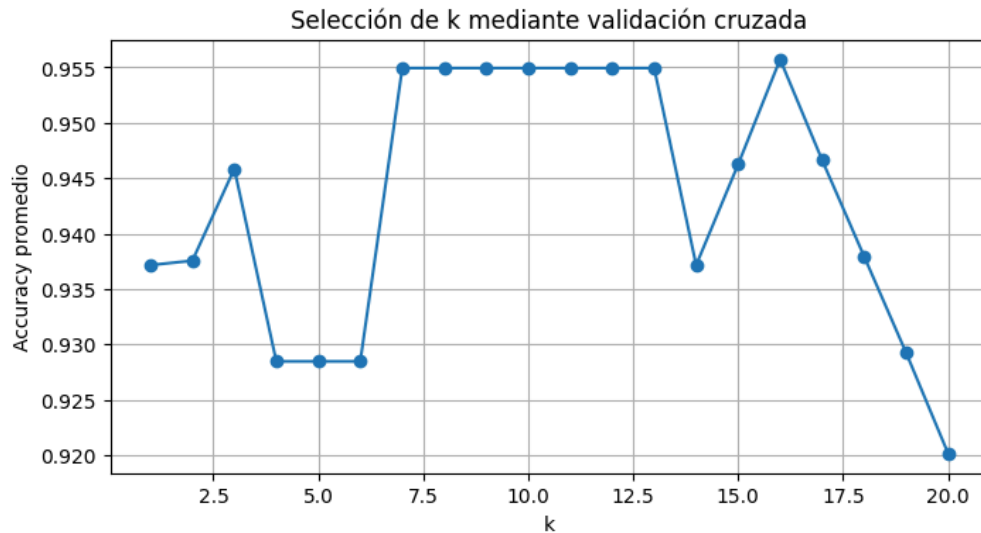
Se estudió el algoritmo K-Nearest Neighbors (KNN), un método de aprendizaje supervisado que clasifica una nueva observación a partir de las clases de sus k vecinos más cercanos. La cercanía se determina mediante una métrica de distancia, normalmente la distancia euclidiana. Debido a que una variable con valores numéricos grandes podría dominar el cálculo de la distancia, las características fueron estandarizadas antes del entrenamiento.

Como aplicación se utilizó el dataset Iris, compuesto por 150 muestras de flores y cuatro características: longitud y ancho del sépal, y longitud y ancho del pétalo. Las muestras pertenecen a tres especies: setosa, versicolor y virginica. El conjunto se dividió en datos de entrenamiento y prueba. Posteriormente se evaluaron valores de k entre 1 y 20 mediante validación cruzada de cinco particiones.

La mejor configuración obtenida fue $k = 16$, con una exactitud promedio de validación de 0.9557, equivalente al 95.57 %. Con este valor se entrenó el modelo definitivo y se evaluó sobre el conjunto de prueba.

Mejor k: 16

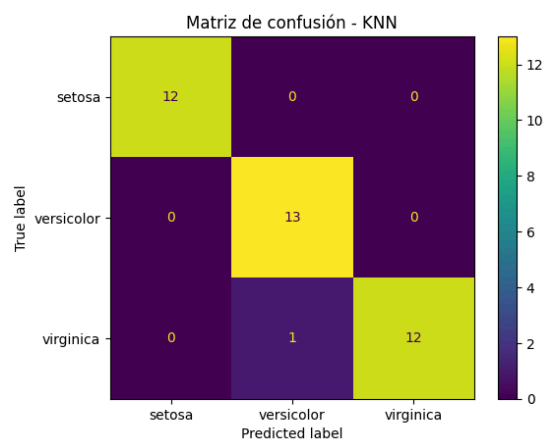
Accuracy CV: 0.9557312252964426



Resultados

El modelo alcanzó una exactitud de 0.9737, es decir, aproximadamente 97.37 %. La matriz de confusión mostró que las 12 muestras de setosa fueron clasificadas correctamente, así como las 13 muestras de versicolor. Para virginica, 12 muestras fueron clasificadas correctamente y una fue confundida con versicolor.

Accuracy prueba: 0.9736842105263158



Esto indica que KNN logró separar muy bien las tres especies y que el único error se produjo entre las dos clases con mayor similitud morfológica.

También se evaluó una flor nueva con medidas de 6.1 cm de longitud de sépalo, 2.8 cm de ancho de sépalo, 4.7 cm de longitud de pétalo y 1.2 cm de ancho de pétalo. El modelo la clasificó como versicolor, con una probabilidad aproximada de 93.75 %, frente a 6.25 % para virginica.

4 . ÁRBOLES DE DECISIÓN

Planteamiento

Se estudió el algoritmo de árboles de decisión, un método supervisado que construye reglas jerárquicas para separar las observaciones en clases. Cada nodo selecciona una característica y un umbral para dividir los datos buscando aumentar la pureza de los grupos resultantes. En este ejercicio se utilizó el índice de Gini como criterio de división.

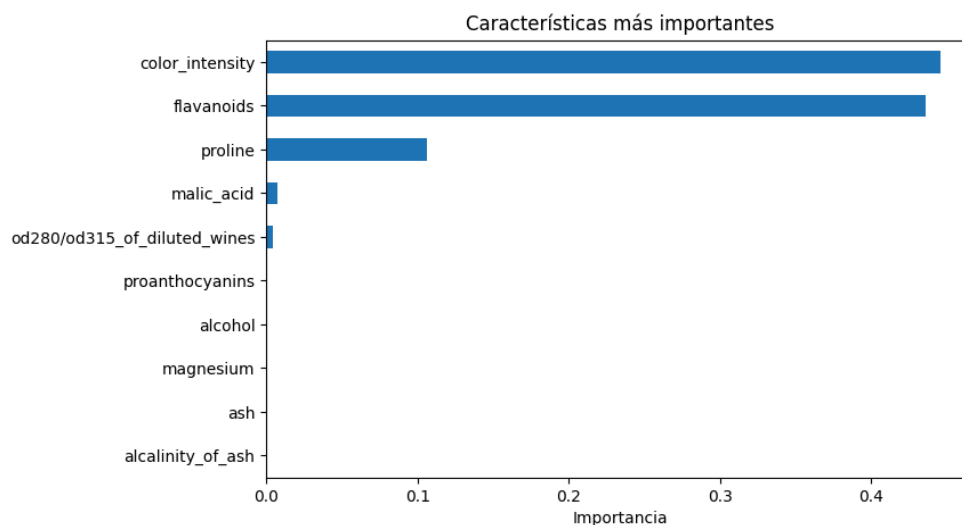
Como aplicación se empleó el dataset Wine de Scikit-learn, que contiene mediciones físico-químicas de diferentes vinos y tres clases objetivo. El conjunto se dividió en entrenamiento y prueba. Para mejorar la interpretación del modelo y reducir el riesgo de sobreajuste se configuró una profundidad máxima de cuatro niveles y un mínimo de tres muestras por hoja.

Resultados



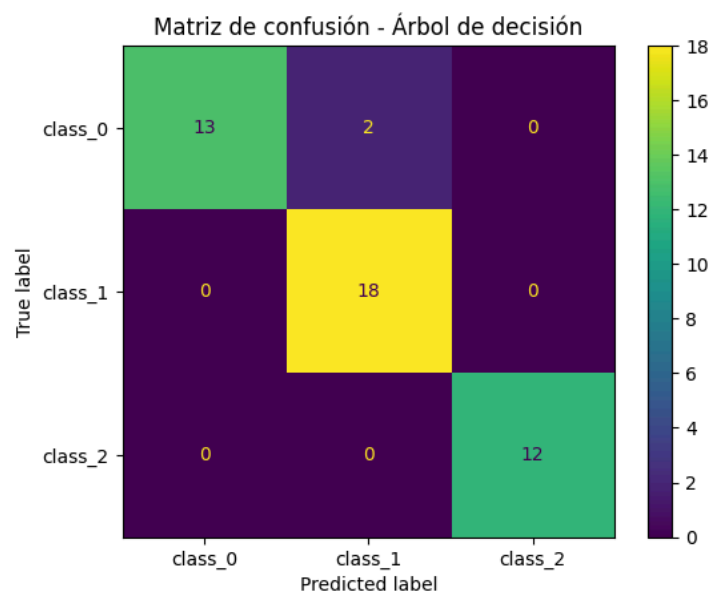
El árbol obtuvo una exactitud de 0.9556, equivalente aproximadamente al 95.56 %. En la matriz de confusión se observa que, de 15 muestras de la clase 0, 13 fueron clasificadas correctamente y 2 se confundieron con la clase 1. Las 18 muestras de la clase 1 y las 12 muestras de la clase 2 fueron clasificadas correctamente.

El análisis de importancia de características mostró que las variables con mayor influencia fueron color_intensity, con una importancia aproximada de 0.4455; flavanoids, con 0.4360; y proline, con 0.1065. En conjunto, estas tres variables concentraron casi toda la capacidad de decisión del árbol.



Clase predicha: class_1

Como prueba adicional se generó una muestra con valores modificados de alcohol, flavonoides e intensidad de color. El modelo la clasificó como class_1.



Estos resultados muestran una de las principales ventajas de los árboles de decisión: además de obtener una alta exactitud, es posible visualizar las reglas y determinar cuáles variables fueron más relevantes para la clasificación.

5 . BAYES INGENUO

Planteamiento

Se estudió el algoritmo Bayes ingenuo, un método probabilístico basado en el teorema de Bayes. El algoritmo estima la probabilidad de pertenencia a cada clase utilizando la probabilidad previa de la clase y la probabilidad de observar las características para dicha clase. Se denomina “ingenuo” porque supone independencia condicional entre las variables de entrada.

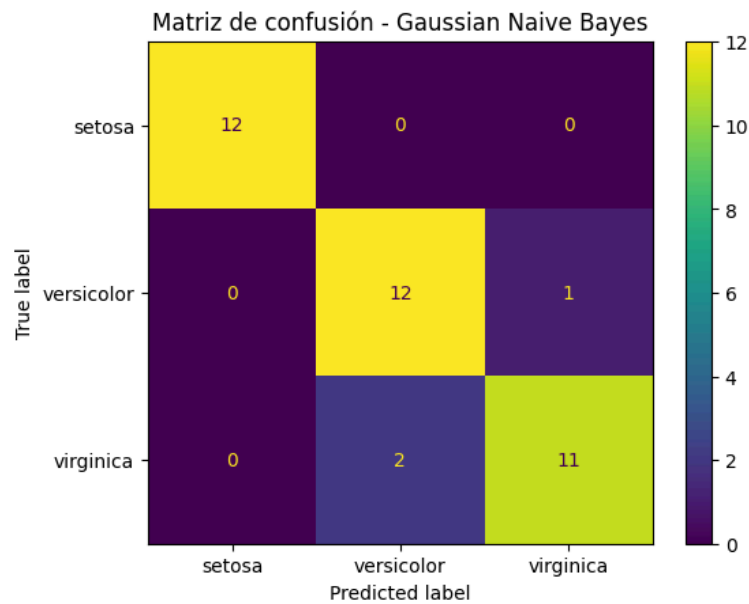
Para el ejemplo se utilizó Gaussian Naive Bayes con el dataset Iris. Esta variante es adecuada para variables continuas porque supone una distribución normal de cada característica dentro de cada clase. El conjunto de datos se dividió en entrenamiento y prueba y posteriormente se entrenó el modelo.

Resultados

El modelo alcanzó una exactitud de 0.9211, equivalente aproximadamente al 92.11 %. En la matriz de confusión, las 12 muestras de setosa fueron clasificadas correctamente. En versicolor se obtuvieron 12 clasificaciones correctas y un error hacia virginica. Para virginica se obtuvieron 11 clasificaciones correctas y dos muestras fueron clasificadas como versicolor, estos son los resultados del código elaborado.

Accuracy prueba: 0.9210526315789473

	precision	recall	f1-score	support
setosa	1.00	1.00	1.00	12
versicolor	0.86	0.92	0.89	13
virginica	0.92	0.85	0.88	13
accuracy		0.92		38
macro avg	0.92	0.92	0.92	38
weighted avg	0.92	0.92	0.92	38



También se evaluó una nueva muestra con medidas de 5.9 cm, 3.0 cm, 5.1 cm y 1.8 cm. El modelo la clasificó como virginica con una probabilidad posterior aproximada de 95.65 %. La probabilidad para versicolor fue de 4.35 %, mientras que para setosa fue prácticamente cero.

El resultado permite observar que Bayes ingenuo no solo entrega una clase final, sino también una estimación probabilística de pertenencia, lo cual facilita interpretar el nivel de confianza del clasificador.

Conclusiones

La regresión polinómica representó adecuadamente la multiplicación, pero su costo no justifica reemplazar una fórmula exacta y sencilla. En la aplicación SVM, RBF permitió capturar relaciones no lineales y detectar más registros de mayor profundidad que la referencia mayoritaria, con numerosos falsos positivos. Ubicación y magnitud no determinan por sí solas la profundidad. La muestra limitada y la posible dependencia entre eventos impiden generalizar estas métricas a otros periodos; se requiere evaluar más datos antes de ampliar su uso.

Referencias

- [1] J. J. Martínez P., Introducción a Machine Learning, material de clase, sep. 2026.
- [2] D. Rodríguez, IA-y-Minirobots, repositorio de código, 2026. <https://github.com/DanielRodriguez2004/IA-y-Minirobots>
- [3] U.S. Geological Survey, Earthquake Catalog API Documentation. <https://earthquake.usgs.gov/fdsnws/event/1/> Consulta: 4 oct. 2026.
- [4] Scikit-learn developers, Support Vector Machines. <https://scikit-learn.org/stable/modules/svm.html> Consulta: 4 oct. 2026.

Se utilizó ChatGPT como apoyo para la organización, redacción y revisión del informe.

Nota: En la elaboración de este documento se utilizó ChatGPT como herramienta de apoyo para la organización, redacción y revisión del contenido. Las fuentes citadas fueron consultadas y verificadas por los autores